



Universidad de Concepción

Tarea 1

Inteligencia Artificial

Asignatura: Inteligencia Artificial

Estudiante: Khristian Daniel Alexander Villalobos Alfaro, matrícula
2024432826

Fecha: 28 de septiembre de 2026

Universidad de Concepción

Índice

1. Introducción	2
2. Dijkstra	2
2.1. Descripción	2
2.2. Análisis	2
3. BFS	3
3.1. Descripción	3
3.2. Análisis	3
4. A*	3
4.1. Descripción	3
4.2. Análisis	3
5. Greedy Best First Search	4
5.1. Descripción	4
5.2. Análisis	4
6. Genetic Algorithm	4
6.1. Descripción	4
6.2. Análisis	4
7. Resultados	5
7.1. Resultados del Algoritmo Dijkstra	5
7.2. Resultados del Algoritmo BFS	7
7.3. Resultados del Algoritmo A*	8
7.4. Resultados del Algoritmo Greedy Best First Search	10
7.5. Resultados del Algoritmo Genetic Algorithm	11
8. Conclusión	12

1. Introducción

En esta tarea se ponen a prueba 5 algoritmos de búsqueda mediante un escenario ficticio en el cual múltiples agentes deben buscar la salida de emergencia antes de ser consumidos por un fuego que se expande cada cierta cantidad de turnos. Los algoritmos puestos a prueba son 2 de búsqueda no informada, 2 de búsqueda informada y 1 genético. Los algoritmos escogidos para esta tarea son los siguientes:

- Dijkstra (no informada)
- BFS (no informada)
- A* (informada)
- GBFS (informada)
- Genético

Cómo información adicional se utilizaron 3 mapas los cuales varían en la cantidad de obstáculos (paredes) que estos tienen. El mapa de mayor densidad tiene pasillos muy estrechos, el mapa de mediana densidad tiene 'habitaciones' conectadas por pequeños pasillos y el mapa de baja densidad presenta obstáculos colocados de manera separada entre sí.

El fuego se propaga a las casillas adyacentes (arriba, abajo, izquierda y derecha) cada 2 turnos que se ejecuta la simulación y este consume a los agentes que se encuentren en dicha casilla.

2. Dijkstra

2.1. Descripción

El primer algoritmo de búsqueda no informada implementado es Dijkstra el cuál busca el camino de menor costo dentro de un grafo con pesos no negativos. Para este caso cada celda adyacente representaría un nodo y el algoritmo considera aquellas celdas las cuales no sean fuego o una pared.

2.2. Análisis

El algoritmo funciona dando un paso hacia el nodo que posee la ruta acumulada más corta, asegurando los resultados a medida que avanza, para ello el algoritmo explora los vecinos con los que tiene conexión directa y calcula cuanto cuesta llegar a ellos sumando el costo del trayecto, luego, el algoritmo actualiza la ruta más corta si el costo calculado para llegar a un vecino es menor que el costo que ese vecino tenía anotado previamente. Una vez se han revisado todos los vecinos el nodo actual se marca como visitado y el algoritmo se desplaza a un nodo que no haya sido visitado y que posea la distancia más corta repitiendo el proceso nuevamente hasta encontrar el nodo objetivo.

Las ventajas de Dijkstra es que asegura encontrar la solución óptima, sin embargo, al ser un algoritmo de búsqueda ciega provoca que su ejecución consuma tiempo y recursos con nodos que se encuentran distantes a el nodo objetivo y si la red es muy compleja y grande este algoritmo podría ser lento en ejecutarse.

3. BFS

3.1. Descripción

Como segundo algoritmo de búsqueda no informada utilizamos BFS el cual expande la búsqueda a todos los nodos cercanos para luego ir expandiéndose por los nodos nuevos, realizando una búsqueda extensa por cada nodo que encuentre cada vez visitando nodos más cercanos. BFS por defecto no tiene pesos en consideración por lo que para adaptarlo a esta tarea la cual exige pesos para celdas con personas se aplica una pequeña modificación, si BFS detecta dos o más caminos con un mismo número de pasos se elige aquel que tenga el menor costo, es decir, únicamente cuando tenemos un empate para el camino más corto es que tenemos en cuenta el peso total.

3.2. Análisis

El algoritmo funciona mediante una cola (Queue) el cual ingresa un nodo y lo marca como visitado y se agrega a la cola, luego, saca el primer nodo de la cola para analizarlo, si es el destino el algoritmo se detiene, si no, revisa los vecinos de dicho nodo. Si alguno de sus vecinos no ha sido visitado lo marcamos como tal y se agrega a la cola repitiendo el proceso reiteradamente.

Al igual que Dijkstra, BFS asegura encontrar solución óptima si esta existe, sin embargo, tiene un gran costo computacional y de memoria.

4. A*

4.1. Descripción

Como primer algoritmo de búsqueda informada tenemos el algoritmo A* el cual es ampliamente utilizado por su rapidez y por garantizar encontrar la solución óptima siempre y cuando la función heurística sea admisible.

4.2. Análisis

A* funciona mediante el uso de una función heurística la cual ayuda a guiar al algoritmo a encontrar la solución óptima y evitar computo en caminos lejanos, esto se hace considerando el costo real más el costo de la heurística. En primer lugar el algoritmo observa los vecinos inmediatos al punto de partida y calcula el valor total para cada uno de ellos, luego elige el mejor de ellos (el valor más bajo) y si este presenta un obstáculo o es inválido el algoritmo busca de los nodos no explorados y elige el que tenga el menor costo. Si el algoritmo encuentra una forma más rápida de llegar a un nodo que ya había visto antes, este actualiza la información para recordar solo la mejor ruta. Este proceso se repite hasta encontrar el nodo objetivo.

La ventaja de A* es que es óptimo y bastante eficiente tanto de forma computacional como en uso de memoria por lo que es el algoritmo más utilizado en muchas aplicaciones y videojuegos que buscan una ruta óptima.

5. Greedy Best First Search

5.1. Descripción

Este es el segundo algoritmo de búsqueda informada el cual es bastante sencillo de implementar ya que se basa únicamente en el uso de la función heurística para encontrar la solución. A diferencia de A^* este algoritmo no asegura encontrar la solución óptima.

5.2. Análisis

Este algoritmo funciona utilizado como función de costo únicamente el valor que otorgue la función heurística. En un comienzo el algoritmo observa los nodos iniciales y selecciona aquel que tenga el menor costo, desplazándose a este nodo, el algoritmo vuelve a consultar la heurística de los nuevos nodos descubiertos y busca de entre todos aquel que tenga el menor costo según la heurística repitiendo así la búsqueda hasta encontrar el nodo objetivo.

6. Genetic Algorithm

6.1. Descripción

Por último tenemos un algoritmo genético el cual está inspirado en la biología evolutiva y buscan encontrar la solución mediante selección natural y mutaciones aleatorias. Para esta tarea se utiliza un algoritmo que cuenta con las fases de población inicial, evaluación (fitness), selección, cruce (crossover), mutación y evolución.

6.2. Análisis

Aquí se detallan los pasos del algoritmo.

- Población inicial: Para el agente que intenta moverse se crea una población inicial de 20 con secuencias aleatorias donde cada cromosoma consiste de 15 movimientos aleatorios que pueden ser arriba, abajo, izquierda, derecha y quedarse quieto.
- Evaluación (fitness): El algoritmo toma esas 20 secuencias de movimientos aleatorias y simula caminarlas en el mapa estático, calculando un "fitness score" para cada secuencia dependiendo de su resultado. Mientras más cerca termine de la salida mayor será su puntaje. Si la secuencia encuentra la salida se le da una puntuación significativa de 1000 puntos. Si la secuencia se intenta mover a una pared o fuego esta es penalizada con -50 puntos. También se descuentan puntos si es que pasa por una celda la cual se encuentra ya un agente en ella.
- Selección y Cruce (crossover): Una vez se hayan evaluado las 20 secuencias el algoritmo elige 3 aleatoriamente y las compara de manera que aquella que tenga la mayor puntuación es escogida como padre y repite esto para tener al segundo padre, una vez se tienen los dos padres, se dividen las secuencias a la mitad y las junta para crear a un hijo el cual tiene los atributos de ambas secuencias.
- Mutación: Existe una probabilidad de un 10 % de que un movimiento individual de un hijo sea cambiado de manera aleatoria a una dirección diferente.

- Evolución: Para evitar perder buenos caminos las mejores dos secuencias con el mayor puntaje de las generaciones previas son traídas a la nueva generación sin cambiarlas.

Este proceso se repite por unas 10 generaciones y al final se selecciona aquella secuencia que tiene el mayor puntaje y el agente realiza la primera acción de dicha secuencia siempre y cuando este sea un movimiento permitido (no camine hacia una pared o fuego)

7. Resultados

En esta sección se presentan los resultados obtenidos al ejecutar los diferentes algoritmos. Para estas pruebas se realizaron 80 iteraciones por algoritmo y mapa (alta densidad, media densidad y baja densidad). Para el número de agentes se utilizaron 80 agentes para Dijkstra, BFS, A* y Greedy Best First Search. Mientras que para el algoritmo genético se utilizaron 20 agentes debido al elevado tiempo de ejecución del algoritmo.

Los mapas son de 50x50 en dimensión con agentes colocados de manera aleatoria dentro del mapa.

7.1. Resultados del Algoritmo Dijkstra

Cuadro 1: Resumen de estadísticas de Dijkstra, mapa de alta densidad.

Métrica	Valor
Estadísticas Finales	
Total de iteraciones jugadas	80
Personas que escaparon	5992 / 6400
Personas que murieron	408 / 6400
Tasa de supervivencia	93.62 %
Distribución del Tiempo de Despeje (Turnos)	
Media de turnos (μ)	948.02
Desviación estándar (σ)	12.86
Mínimo de turnos	898
Máximo de turnos	957

Cuadro 2: Resumen de estadísticas de Dijkstra, mapa de media densidad.

Métrica	Valor
Estadísticas Finales	
Total de iteraciones jugadas	80
Personas que escaparon	6400 / 6400
Personas que murieron	0 / 6400
Tasa de supervivencia	100.00 %
Distribución del Tiempo de Despeje (Turnos)	
Media de turnos (μ)	91.72
Desviación estándar (σ)	4.20
Mínimo de turnos	78
Máximo de turnos	101

Cuadro 3: Resumen de estadísticas de Dijkstra, mapa de baja densidad.

Métrica	Valor
Estadísticas Finales	
Total de iteraciones jugadas	80
Personas que escaparon	6400 / 6400
Personas que murieron	0 / 6400
Tasa de supervivencia	100.00 %
Distribución del Tiempo de Despeje (Turnos)	
Media de turnos (μ)	96.66
Desviación estándar (σ)	4.06
Mínimo de turnos	85
Máximo de turnos	108

7.2. Resultados del Algoritmo BFS

Cuadro 4: Resumen de estadísticas para BFS, mapa de alta densidad.

Métrica	Valor
Estadísticas Finales	
Total de iteraciones jugadas	80
Personas que escaparon	5973 / 6400
Personas que murieron	427 / 6400
Tasa de supervivencia	93.33 %
Distribución del Tiempo de Despeje (Turnos)	
Media de turnos (μ)	946.00
Desviación estándar (σ)	11.32
Mínimo de turnos	910
Máximo de turnos	957

Cuadro 5: Resumen de estadísticas para BFS, mapa de media densidad.

Métrica	Valor
Estadísticas Finales	
Total de iteraciones jugadas	80
Personas que escaparon	6400 / 6400
Personas que murieron	0 / 6400
Tasa de supervivencia	100.00 %
Distribución del Tiempo de Despeje (Turnos)	
Media de turnos (μ)	88.46
Desviación estándar (σ)	3.55
Mínimo de turnos	80
Máximo de turnos	93

Cuadro 6: Resumen de estadísticas para BFS, mapa de baja densidad.

Métrica	Valor
Estadísticas Finales	
Total de iteraciones jugadas	80
Personas que escaparon	6400 / 6400
Personas que murieron	0 / 6400
Tasa de supervivencia	100.00 %
Distribución del Tiempo de Despeje (Turnos)	
Media de turnos (μ)	87.71
Desviación estándar (σ)	3.64
Mínimo de turnos	78
Máximo de turnos	93

7.3. Resultados del Algoritmo A*

Cuadro 7: Resumen de estadísticas para A*, mapa de alta densidad.

Métrica	Valor
Estadísticas Finales	
Total de iteraciones jugadas	80
Personas que escaparon	5991 / 6400
Personas que murieron	409 / 6400
Tasa de supervivencia	93.61 %
Distribución del Tiempo de Despeje (Turnos)	
Media de turnos (μ)	947.84
Desviación estándar (σ)	11.98
Mínimo de turnos	887
Máximo de turnos	957

Cuadro 8: Resumen de estadísticas para A*, mapa da mediana densidad.

Métrica	Valor
Estadísticas Finales	
Total de iteraciones jugadas	80
Personas que escaparon	6400 / 6400
Personas que murieron	0 / 6400
Tasa de supervivencia	100.00 %
Distribución del Tiempo de Despeje (Turnos)	
Media de turnos (μ)	89.35
Desviación estándar (σ)	3.33
Mínimo de turnos	81
Máximo de turnos	97

Cuadro 9: Resumen de estadísticas para A*, mapa de baja densidad.

Métrica	Valor
Estadísticas Finales	
Total de iteraciones jugadas	80
Personas que escaparon	6400 / 6400
Personas que murieron	0 / 6400
Tasa de supervivencia	100.00 %
Distribución del Tiempo de Despeje (Turnos)	
Media de turnos (μ)	88.22
Desviación estándar (σ)	3.15
Mínimo de turnos	80
Máximo de turnos	93

7.4. Resultados del Algoritmo Greedy Best First Search

Cuadro 10: Resumen de estadísticas de Greedy, mapa de alta densidad.

Métrica	Valor
Estadísticas Finales	
Total de iteraciones jugadas	80
Personas que escaparon	5919 / 6400
Personas que murieron	481 / 6400
Tasa de supervivencia	92.48 %
Distribución del Tiempo de Despeje (Turnos)	
Media de turnos (μ)	946.71
Desviación estándar (σ)	12.60
Mínimo de turnos	877
Máximo de turnos	957

Cuadro 11: Resumen de estadísticas de Greedy, mapa de mediana densidad.

Métrica	Valor
Estadísticas Finales	
Total de iteraciones jugadas	80
Personas que escaparon	6400 / 6400
Personas que murieron	0 / 6400
Tasa de supervivencia	100.00 %
Distribución del Tiempo de Despeje (Turnos)	
Media de turnos (μ)	121.90
Desviación estándar (σ)	5.38
Mínimo de turnos	105
Máximo de turnos	129

Cuadro 12: Resumen de estadísticas de Greedy, mapa de baja densidad.

Métrica	Valor
Estadísticas Finales	
Total de iteraciones jugadas	80
Personas que escaparon	6400 / 6400
Personas que murieron	0 / 6400
Tasa de supervivencia	100.00 %
Distribución del Tiempo de Despeje (Turnos)	
Media de turnos (μ)	88.29
Desviación estándar (σ)	3.03
Mínimo de turnos	81
Máximo de turnos	93

7.5. Resultados del Algoritmo Genetic Algorithm

Cuadro 13: Resumen de estadísticas para Genetic, mapa de alta densidad.

Métrica	Valor
Estadísticas Finales	
Total de iteraciones jugadas	80
Personas que escaparon	191 / 1600
Personas que murieron	1409 / 1600
Tasa de supervivencia	11.94 %
Distribución del Tiempo de Despeje (Turnos)	
Media de turnos (μ)	1695.75
Desviación estándar (σ)	87.76
Mínimo de turnos	1181
Máximo de turnos	1737

Cuadro 14: Resumen de estadísticas para Genetic, mapa de mediana densidad.

Métrica	Valor
Estadísticas Finales	
Total de iteraciones jugadas	80
Personas que escaparon	1052 / 1600
Personas que murieron	548 / 1600
Tasa de supervivencia	65.75 %
Distribución del Tiempo de Despeje (Turnos)	
Media de turnos (μ)	171.94
Desviación estándar (σ)	3.65
Mínimo de turnos	161
Máximo de turnos	178

Cuadro 15: Resumen de estadísticas de Genetic, mapa de baja densidad.

Métrica	Valor
Estadísticas Finales	
Total de iteraciones jugadas	80
Personas que escaparon	1600 / 1600
Personas que murieron	0 / 1600
Tasa de supervivencia	100.00 %
Distribución del Tiempo de Despeje (Turnos)	
Media de turnos (μ)	86.38
Desviación estándar (σ)	5.73
Mínimo de turnos	71
Máximo de turnos	98

8. Conclusión

Con estos resultados se puede notar la diferencia que ocurre entre cada tipo de algoritmo al igual que sus ventajas y desventajas. Notablemente se puede observar como los algoritmos de búsqueda informada son, por lo general, más eficientes que los algoritmos de búsqueda no informada, sin embargo, en el caso del mapa de alta densidad se puede observar que la búsqueda informada puede arrojar peores resultados debido a que la función heurística puede guiar al algoritmo a caminos sin salidas los cuales ralentizan su progreso.

También se puede notar como el algoritmo genético requiere considerablemente mayor poder de computo y sus resultados suelen ser más variados debido a la naturaleza de estos los cuales dependen de ciertos factores de aleatoriedad y requieren de muchas iteraciones para converger a una solución. En definitiva el algoritmo ideal para resolver un problema depende significativamente del objetivo, los recursos computacionales disponibles y el número de agentes los cuales estarán actuando simultáneamente.

Sobre el uso de apoyo externo en la creación de esta tarea se declara que se utilizaron

herramientas de inteligencia artificial en la creación de mapas y como apoyo en la corrección y correctitud de los algoritmos implementados.